

Important

Here are some guidelines you should follow for this homework. Note that grade penalties may apply if you do not follow these guidelines.

1. All submitted code must compile under **JDK 25**. This includes unused code, so don't submit extra files that don't compile. Any compile errors will result in a 0.
2. Do not include any package declarations in your classes.
3. Do not change any existing class headers, constructors, instance/global variables, or method signatures. For example, do not add **throws** to the method headers since they are not necessary.
4. Do not add additional public methods.
5. Do not use anything that would trivialize the assignment. (e.g. Don't import/use `java.util.ArrayList` for an `ArrayList` assignment. Ask if you are unsure.)
6. Always be very conscious of efficiency. Even if your method is to be $O(n)$, traversing the structure multiple times is considered inefficient unless that is absolutely required (and that case is extremely rare).
7. You are expected to implement all methods on the homework. Each unimplemented method will result in a deduction.
8. You must submit your source code, the `.java` files, not the compiled `.class` files.
9. Only the active submission will be graded. Make sure your active submission has **all** required files.
10. After you submit your files, redownload them and run them to make sure they are what you intended to submit. You are responsible if you submit the wrong files.

Collaboration Policy

Every student is expected to read, understand and abide by the [Georgia Tech Academic Honor Code](#).

When working on homework assignments, you **may not** directly copy code from any source (other than your own past submissions). You are welcome to collaborate with peers and consult external resources, but you **must** personally write all of the code you submit. **You must list, at the top of each file in your submission, every student with whom you collaborated and every resource you consulted while completing the assignment.**

You may not directly share any files containing assignment code with other students or post your code publicly online. If you wish to store your code online in a personal **private** repository, you can use [GitHub Enterprise](#) to do this for free.

Violators of the collaboration policy for this course will be turned into the Office of Student Integrity.

Style and Formatting

It is important that your code is not only functional, but written clearly and with good programming style. Your code will be checked against a style checker. The style checker is provided to you, and is located on Canvas. It can be found under Files, along with instructions on how to use it. A point is deducted for every style error that occurs. If there is a discrepancy between what you wrote in accordance with good style and the style checker, then address your concerns with a Head TA.

Javadocs

You should add Javadocs to any helper methods you create in a style similar to the existing Javadocs. Any Javadocs you write must be useful and describe the contract, parameters, and return value of the method. Random or useless Javadocs added only to appease Checkstyle will lose points.

Vulgar/Obscene Language

Any submission that contains profanity, vulgar, or obscene language will receive an automatic zero on the assignment. This policy applies not only to comments/Javadocs, but also things like variable names.

Exceptions

When throwing exceptions, you must include a message by passing in a String as a parameter. **The message must be useful and tell the user what went wrong.** “Error”, “BAD THING HAPPENED”, and “fail” are not good messages. The name of the exception itself is not a good message.

For example:

Bad: `throw new IndexOutOfBoundsException("Index is out of bounds.");`

Good: `throw new IllegalArgumentException("Cannot insert null data into data structure.");`

In addition, you may not use try-catch blocks to catch an exception unless you catching an exception you have explicitly thrown yourself with the `throw new ExceptionName("Exception Message");` syntax (replacing `ExceptionName` and `Exception Message` with the actual exception name and message respectively).

Generics

If available, use the generic type of the class; do **not** use the raw type of the class. For example, use `new LinkedList<Integer>()` instead of `new LinkedList()`. Using the raw type of the class will result in a penalty.

Forbidden Statements

You may not use these in your code code for this homework assignment.

- `package`
- `System.arraycopy()`
- `clone()`
- `assert()`
- `Arrays` class
- `Array` class
- `Thread` class
- `Collections` class
- `Collection.toArray()`
- Reflection APIs
- Inner or nested classes
- Lambda Expressions

- Method References (using the `::` operator to obtain a reference to a method)

If you're not sure on whether you can use something, and it's not mentioned here or anywhere else in the homework files, just ask.

Debug print statements are fine, but nothing should be printed when we run your code. We expect clean runs—printing to the console when we're grading may result in a penalty.

JUnits

We have provided a **very basic** set of tests for your code. These tests do not guarantee the correctness of your code (by any measure), nor do they guarantee you any grade. You are encouraged to write your own tests to help verify correctness. For all homeworks, do **NOT** post your tests on the public GitHub or any other public source.

If you need help on running JUnits, there is a guide, available on Canvas under Files, to help you run JUnits on the command line or in IntelliJ.

Sorting

For this assignment you will be coding 3 different algorithms: merge sort, LSD radix sort, and K^{th} Select. In addition to the requirements for each sort, to test for efficiency, we will be looking at the number of comparisons made between elements while grading.

For each of the sorting algorithms, you may assume that the arrays you are sorting will not contain null. You should also assume that arrays may contain any number of duplicate elements.

Your implementations must match what was taught in lecture and recitation to receive credit. Implementing a different sort or a different implementation for a sort will receive no credit even if it passes comparison checks.

Comparator

Each method (except radix sort) will take in a Comparator and use it to compare the elements of the array in various algorithms described below and in the sorting file. You **must** use this Comparator as the number of comparisons performed with it will be used when testing your assignment. See the Java API for details about how the Comparator works and the meaning of the returned value.

Generic Methods

Most of the assignments for this class so far have utilized generics by incorporating them into the class declaration. However, this assignment will have you implement various algorithms as static methods in a utility class. Thus, the generics from here on will use generic methods instead of generic classes (hence the `<T>` in each of the method headers and javadocs). This also means any helper methods you create will also need to be static with the same `<T>` in the method header.

In-Place Sorts

Some of the sorts below are in-place sorts. This means that the items in the array passed in **should not** get copied over to another data structure. Note that you can still create variables that hold only one item; you cannot create another data structure such as an array or list in the method.

Stable Sorts

Some of the sorts below are stable sorts. This means that duplicates **must** remain in the same relative positions after sorting as they were before sorting.

Adaptive Sorts

Some of the sorts below are adaptive. This means that the algorithm takes advantage of existing order in the input array by not comparing elements that are already ordered.

Algorithms

Merge Sort

Merge sort should be out-of-place, stable, and not adaptive. It should have a worst case running time of $O(n \log n)$ and a best case running time of $O(n \log n)$. When splitting an odd size array, the extra data should go on the **right**.

K^{th} Select

K^{th} Select should be in-place. It should have a worst case running time of $O(n^2)$ and a best case running time of $O(n)$. Your implementation of pivot selection must be randomized as specified in the method's Javadocs. Logically, it is similar to a one-sided Quicksort. When asked for the k^{th} smallest value, you should return what would be at index $k - 1$ if the array was perfectly sorted in ascending order.

LSD Radix Sort

LSD Radix sort should be out-of-place, stable, and not adaptive. It should have a worst case running time of $O(kn)$ and a best case running time of $O(kn)$, where k is the number of digits in the longest number. You will be implementing the least significant digit version of the sort. You will be sorting **ints**. Note that you CANNOT change the **ints** into **Strings** at any point in the sort for this exercise. The sort **must** be done in base 10. Also, as per the forbidden statements section, you cannot use anything from the **Math** class besides **Math.abs()**. However, be wary of handling overflow if you use **Math.abs()**!

Grading

Here is the grading breakdown for the assignment. There are various deductions not listed that are incurred when breaking the rules listed in this PDF, and in other various circumstances.

Methods:	
mergeSort	25pts
kthSelect	25pts
lsdRadixSort	25pts
Other:	
Checkstyle	10pts
Efficiency	15pts
Total:	100pts

Provided

The following file(s) have been provided to you. There are several, but we've noted the ones to edit.

1. `Sorting.java`

This is the class in which you will implement the different sorting algorithms. Feel free to add private helper methods but **do not add any new public methods, inner/nested classes, instance variables, or static variables**.

2. `StudentTest.java`

This is the test class that contains a set of tests covering the basic algorithms in the **Sorting** class. It is not intended to be exhaustive and does not guarantee any type of grade. **Write your own tests to ensure you cover all edge cases.**

Deliverables

You must submit the following file. Make sure the filename matches the filename below, and that *only* the following file is present. Do not zip up the file before submitting.

Once submitted, double check that it has uploaded properly on Gradescope. To do this, download your uploaded file to a new folder, copy over the support file(s), recompile, and run. It is your sole responsibility to re-test your submission and discover editing oddities, upload issues, etc.

1. `Sorting.java`

Autograder

The autograder for this homework will give you some helpful information, but it will be limited. Every time you submit, you will receive info about any compiler errors, Checkstyle deductions, and general formatting issues. The **first three times per day** you submit, you will see your per-method score. You will not see the number of failed tests or any information about the tests themselves.

Note that while this score is generally representative, **it is not final**. We go over each submission manually, and it is possible for autograder points earned to be deducted in certain cases, including, but not limited to, attempting to brute-force test cases. Additionally, we will manually grade efficiency, so that score is not included in the autograder score. Finally, **we reserve the right to update our test cases** should any defects be discovered, which may result in slight changes to your autograder score.